



15

16 `return blurred, edge_strength`

## 1.3 实验结果及分析

- 高斯滤波效果：
  - **平滑程度**：随着窗口大小和  $\sigma$  的增加，图像变得更加平滑，高频细节逐渐模糊。
  - **极端情况**：小窗口 ( $3 \times 3$ ) 和小尺度 ( $\sigma = 1$ ) 的组合保留了更多图像细节；而大窗口 ( $11 \times 11$ ) 和大尺度 ( $\sigma = 7$ ) 的组合产生了最明显的平滑效果，类似于“毛玻璃”效果。
- LoG 边缘检测效果：
  - **Sigma ( $\sigma$ ) 的影响**：
    - $\sigma$  较小 (如 1) 时：检测到精细的边缘和纹理细节，但也包含较多噪声。
    - $\sigma$  较大 (如 7) 时：细微纹理被平滑，仅保留主要轮廓 (如帽子、脸部轮廓)，且边缘线条变宽。
  - **窗口大小的影响**：窗口大小需大于  $3\sigma$  才能近似完整的高斯分布。大窗口能提供更好的抗噪能力，但计算量增加。
  - **彩色与灰度对比**：彩色图像的边缘检测结果通常比灰度图像更丰富，因为它利用了 RGB 三个通道的色彩变化信息。

## 第二部分：Task 2 - Canny 检测子

### 2.1 实验内容

- **数据**：Lena 灰度图。
- **目标**：使用 Canny 算子检测边缘，探究双阈值 (Dual-Thresholding) 对检测结果的影响。
- **参数设置**：测试了多组  $(T_1, T_2)$  阈值组合，如  $(50, 150)$ ,  $(100, 200)$ ,  $(30, 100)$ ,  $(100, 120)$ ,  $(50, 250)$ 。

### 2.2 实验原理与代码实现

原理：

Canny 算法被认为是最优边缘检测算法之一，包含四个步骤：

1. **高斯平滑**：去噪。
2. **梯度计算**：计算幅值和方向。
3. **非极大值抑制 (NMS)**：细化边缘，保留局部最大值。
4. **双阈值连接**：
  - 强边缘 ( $> T_2$ )：直接保留。
  - 弱边缘 ( $T_1 < val < T_2$ )：仅在连接到强边缘时保留 (边缘滞后技术)。

关键代码实现 (Task2/task2.py)：

```
1 # 定义参数列表：(低阈值, 高阈值)
2 params = [
3     (50, 150), # 标准推荐比例 (1:3)
4     (100, 200), # 高阈值, 只检测强边缘
5     (30, 100), # 低阈值, 保留更多细节
```

```

6         (100, 120), # 窄范围
7         (50, 250)   # 宽范围
8     ]
9
10    for t1, t2 in params:
11        # 自动包含: 高斯平滑 -> 梯度计算 -> NMS -> 双阈值连接
12        edges = cv2.Canny(img, t1, t2)
13        filename = f"lena_canny_t1_{t1}_t2_{t2}.png"
14        cv2.imwrite(os.path.join(output_dir, filename), edges)

```

## 2.3 实验结果及分析

通过对比不同阈值组合的结果：

1. **标准阈值 (50, 150)**：结果最为均衡。人物轮廓清晰，发丝等细节保留适中，且背景噪声较少，是较好的默认选择。
2. **高阈值 (100, 200)**：抑制了大量弱边缘，丢失了面部纹理和大部分头发细节，仅保留了最显著的几何轮廓（如帽子外沿）。
3. **低阈值 (30, 100)**：检测出大量边缘，细节极其丰富，但也引入了大量噪声和非显著纹理（如皮肤微小瑕疵），图像显得杂乱。
4. **窄范围 (100, 120)**：由于高低阈值接近，边缘连接的效果减弱，结果介于高阈值和低阈值之间，部分弱边缘可能断裂。
5. **宽范围 (50, 250)**：高阈值很高导致强边缘较少，但低阈值较低允许连接更多弱边缘。这要求强边缘必须足够显著才能作为“种子”延伸出弱边缘。

**结论：** Canny 算子对阈值非常敏感。实际应用需根据图像对比度和任务需求权衡：若需提取物体轮廓则提高阈值，若需分析纹理细节则降低阈值。

## 第三部分：Task 3 - BSDS500 数据集评估

### 3.1 实验内容

- **数据：** BSDS500 数据集（测试集）。
- **目标：** 量化评估 Canny 算法在复杂自然场景下的边缘检测性能。
- **指标：**
  - **精确率 (Precision):**  $TP / (TP + FP)$ ，检测准不准。
  - **召回率 (Recall):**  $TP / (TP + FN)$ ，检测全不全。

### 3.2 实验原理与代码实现

**原理：**

由于算法检测的边缘与人工标注（Ground Truth）可能存在微小位移，直接像素匹配不合理。评估时引入**容差机制**：对二值化的边缘图进行形态学**膨胀 (Dilation)**，允许 1-2 像素的误差。

**关键代码实现** (Task3/task3.py)：

```

1 # 容差核: 3x3 的全 1 矩阵, 允许 1 像素的误差
2 kernel = np.ones((3,3), np.uint8)
3
4 # ... (循环处理每张图) ...

```

```
5
6 # 1. 准备工作：分别膨胀检测结果和 GT
7 dilated_detected = cv2.dilate(detected_edges_bin, kernel)
8 dilated_gt = cv2.dilate(gt_boundary, kernel)
9
10 # 2. 计算 Precision
11 # 分子：检测出的边缘中，能匹配到膨胀后 GT 的像素数
12 tp_prec = np.sum(detected_edges_bin & dilated_gt)
13 n_detected = np.sum(detected_edges_bin)
14 precision = tp_prec / (n_detected + 1e-8)
15
16 # 3. 计算 Recall
17 # 分子：真实 GT 边缘中，能被膨胀后检测结果覆盖的像素数
18 tp_rec = np.sum(gt_boundary & dilated_detected)
19 n_gt = np.sum(gt_boundary)
20 recall = tp_rec / (n_gt + 1e-8)
```

### 3.3 实验结果及分析

对 BSDS500 测试集所有图片取平均值：

- **Global Average Precision: 0.1564**
- **Global Average Recall: 0.7113**

分析：

1. **高召回率 (71%)**：说明 Canny (阈值 100/200) 能够成功捕获图像中大部分的显著边缘，漏检情况较少。
2. **低精确率 (15%) - 语义鸿沟**：
  - **算法视角**：Canny 关注像素级梯度，将草丛、地毯花纹等纹理都视为边缘。
  - **人类视角**：Ground Truth 通常只标记物体级轮廓（语义边缘）。
  - **结论**：大量被算法检测为边缘的纹理细节在 Ground Truth 中未被标记，导致误检率（False Positive）极高。
3. **改进方向**：
  - 参数调优：针对不同图像自适应调整阈值。
  - 后处理：增加纹理抑制步骤。
  - 算法升级：使用基于深度学习的边缘检测模型（如 HED, RCF）来缩小语义鸿沟。